

Hashing + Intervals Pattern in DSA

Below are detailed textbook-style notes on the Hashing + Intervals DSA pattern, with emphasis on pattern recognition, invariants, complexity, edge cases, and Google-style interview follow-ups.

1. Why This Pattern Matters

Two ideas appear repeatedly in interview problems:

1. Hashing — efficiently remember information you've already seen.
2. Intervals — reason about ranges of values or time and determine whether they overlap, merge, contain one another, or can coexist.

They are often combined because interval problems frequently require you to remember:

- which endpoints have appeared,
- which ranges are active,
- frequencies,
- previously processed intervals,
- or whether a particular value/range has already been encountered.

The key mental model is:

Hashing gives you fast lookup; intervals give you structure over ranges.

PART I — HASHING

2. What Is Hashing?

Hashing is a technique for storing data in a structure that allows approximately $O(1)$ average-time lookup, insertion, and deletion.

In Python, the primary structures are:

```
seen = set()

for x in nums:
    if x in seen:
        return True
    seen.add(x)

return False
```

The important idea is not the Python syntax.

It is:

```
Have I seen this before?
  ↓
Hash
  ↓
 $O(1)$  average lookup
```

3. Why Hashing Is Powerful in DSA

Consider:

```
nums = [4, 7, 2, 9, 7]
```

Suppose the question is:

Does any value occur twice?

A brute-force approach compares every pair.

For n elements:

$O(n^2)$

Hashing lets us remember previously encountered values:

```
4 → seen
7 → seen
2 → seen
9 → seen
7 → already seen
```

Complexity:

```
Time:  $O(n)$  average
Space:  $O(n)$ 
```

This is one of the most important transformations in interview problem solving:

```
Repeated searching
↓
Remember previous information
↓
Hash set/map
↓
 $O(n)$  average solution
```

4. Hash Set vs Hash Map

Hash Set

Use a set when you mainly care about:

Does this value exist?

Example:

```
seen = set()

for x in nums:
    if x in seen:
        ...
    seen.add(x)
```

Typical applications:

- duplicate detection

- membership
- visited nodes
- unique values
- cycle-related state

Hash Map

Use a dictionary when you need:

value → associated information

For example:

```
frequency = {}
for x in nums:
    frequency[x] = frequency.get(x, 0) + 1
```

Now:

value → frequency

Example:

```
"a" → 3
"b" → 2
"c" → 1
```

5. The Most Important Hashing Question

When reading a problem, ask:

What information would I like to remember from earlier elements?

This question often reveals the hash structure immediately.

Examples:

Duplicate detection

Remember:

values seen

→ set

Frequency problem

Remember:

value → count

→ dict

Two Sum

Remember:

```
value → index
```

→ dict

Group Anagrams

Remember:

```
character signature → list of words
```

→ dict

Prefix Sum

Remember:

```
prefix_sum → frequency/index
```

→ dict

6. Hashing Pattern: Frequency Map

Consider:

```
nums = [1, 2, 2, 3, 1, 1]
```

Build:

```
freq = {}
```

```
for x in nums:
    freq[x] = freq.get(x, 0) + 1
```

Result:

```
1 → 3
2 → 2
3 → 1
```

The pattern is:

```
Input
↓
Traverse once
↓
Update dictionary
↓
Use accumulated information
```

7. Hashing Pattern: Lookup for Complement

Classic example:

Two Sum

Given:

```
nums = [2, 7, 11, 15]
target = 9
```

For each number:

```
required = target - current
```

At 2:

```
9 - 2 = 7
```

Ask:

Have I already seen 7?

No.

Store:

```
2 → index 0
```

At 7:

```
9 - 7 = 2
```

2 exists.

Therefore:

```
[0, 1]
```

The important pattern is:

```
Current value
↓
Calculate required value
↓
Hash lookup
↓
Found?
```

This pattern appears in many variations.

8. Hashing Pattern: Prefix Sum + Hash Map

This is particularly important.

Suppose:

```
nums = [1, 2, 3]
```

Prefix sums:

index	0	1	2
nums	1	2	3
prefix	1	3	6

If:

$$\text{prefix}[j] - \text{prefix}[i] = k$$

then:

$$\text{sum}(i+1 \dots j) = k$$

Therefore, when the current prefix sum is S , we look for:

$$S - k$$

in our hash map.

This produces the powerful pattern:

Prefix Sum
+
Hash Map
↓
Subarray problems

9. Hashing and Time Complexity

Hash tables generally provide:

Operation • Average • Worst case

Search • $O(1)$ • $O(n)$

Insert • $O(1)$ • $O(n)$

Delete • $O(1)$ • $O(n)$

For interview analysis, normally state:

$O(1)$ average-case lookup

rather than blindly saying $O(1)$.

For n elements:

n hash operations
→ $O(n)$ average

10. Hashing Trade-Off

Hashing usually trades:

Space

for:

Time

Example:

Brute force:

$O(n^2)$ time
 $O(1)$ space

Hashing:

$O(n)$ average time
 $O(n)$ space

This is a fundamental interview trade-off.

PART II — INTERVALS

11. What Is an Interval?

An interval represents a continuous range.

For example:

[2, 5]

means:

$2 \leq x \leq 5$

In programming interview problems, an interval is usually represented as:

[start, end]

Example:

[2, 5]
[7, 10]
[12, 15]

12. Visualizing Intervals

Suppose:

A = [1, 5]
B = [3, 7]

Visual representation:

A: 1 ----- 5
B: 3 ----- 7

They overlap.

The overlap is:

[3, 5]

Their union is:

[1, 7]

13. The First Question in Interval Problems

Whenever you see intervals, immediately ask:

Are the intervals sorted?

If not:

```
intervals.sort()
```

is often the first step.

Why?

Because sorting creates an ordering that lets you reason locally.

Instead of comparing every interval with every other interval:

$O(n^2)$

you can often process them sequentially:

$O(n \log n)$

because of sorting.

This is one of the most important interval patterns.

14. Interval Sorting

Suppose:

```
intervals = [  
    [5, 7],  
    [1, 3],  
    [2, 6],  
    [8, 10]  
]
```

Sort by start:

```
intervals.sort()
```

Result:

```
[1, 3]  
[2, 6]  
[5, 7]  
[8, 10]
```

Now the intervals are ordered by their starting points.

This dramatically simplifies reasoning.

15. Interval Overlap

For two intervals:

```
A = [a_start, a_end]
B = [b_start, b_end]
```

They overlap if:

```
a_start <= b_end
AND
b_start <= a_end
```

Equivalent interpretation:

They do not overlap if one is completely to the left of the other.

```
A: ----
B:          ----
```

or:

```
B: ----
A:          ----
```

So:

```
No overlap:
a_end < b_start
OR
b_end < a_start
```

Therefore:

```
Overlap:
NOT(no overlap)
```

16. Important Boundary Question

Consider:

```
[1, 5]
[5, 8]
```

Do these overlap?

The answer depends on the problem's interval convention.

If intervals are closed:

```
[1, 5]
[5, 8]
```

they share point 5.

If the problem treats touching intervals as mergeable, they should merge:

```
[1, 8]
```

But some scheduling problems consider an interval ending at time 5 compatible with another starting at 5.

Therefore always ask:

Does touching count as overlap?

This is one of the most common interval edge cases.

17. Merge Intervals Pattern

This is the canonical interval problem.

Input:

```
[1,3]
[2,6]
[8,10]
[9,12]
```

First sort:

```
[1,3]
[2,6]
[8,10]
[9,12]
```

Start with:

```
[1,3]
```

Compare next interval:

```
[2,6]
```

Since:

```
2 <= 3
```

they overlap.

Merge:

```
[1,6]
```

Next:

```
[8,10]
```

Since:

```
8 > 6
```

there is no overlap.

Start a new interval:

```
[8,10]
```

Next:

```
[9,12]
```

Overlap:

```
9 <= 10
```

Merge:

[8,12]

Result:

[1,6]
[8,12]

18. Merge Intervals Algorithm

Conceptually:

```
intervals.sort()

merged = []

for start, end in intervals:
    if not merged or start > merged[-1][1]:
        merged.append([start, end])
    else:
        merged[-1][1] = max(merged[-1][1], end)
```

The critical invariant is:

merged contains non-overlapping intervals representing all intervals processed so far.

When the current interval overlaps the last merged interval:

```
merged[-1][1] = max(merged[-1][1], end)
```

Why max?

Because the new interval may extend farther.

Example:

```
Existing: [1, 10]
Current:  [2, 5]
```

The merged interval must remain:

[1,10]

not:

[1,5]

19. Complexity of Merge Intervals

Sorting:

$O(n \log n)$

Single traversal:

$O(n)$

Total:

$O(n \log n)$

Space:

$O(n)$

for the output.

Depending on the implementation and sorting behavior, auxiliary space analysis can vary, so distinguish:

- output space
- auxiliary algorithmic space

in interviews.

20. Insert Interval

Another major pattern.

Suppose existing intervals are:

[1, 3]
[6, 9]

Insert:

[2, 5]

Result:

[1, 5]
[6, 9]

The process has three conceptual regions:

1. Intervals completely before new interval
2. Intervals overlapping new interval
3. Intervals completely after new interval

21. Insert Interval Mental Model

Given:

`new = [start, end]`

Phase 1 — Before

If:

`current_end < new_start`

the current interval is completely before the new interval.

Add it unchanged.

Phase 2 — Overlap

While:

```
current_start <= new_end
```

merge:

```
new_start = min(new_start, current_start)
new_end = max(new_end, current_end)
```

Phase 3 — After

Once:

```
current_start > new_end
```

the remaining intervals are after the merged interval.

Append them.

22. Non-Overlapping Intervals

Another common interval problem asks:

How many intervals must be removed so that the remaining intervals do not overlap?

This is a greedy problem.

The key insight is:

When intervals overlap, keep the interval that ends earlier.

Why?

Suppose:

```
A = [1, 10]
B = [2, 3]
```

Keeping A blocks many future intervals.

Keeping B finishes at:

```
3
```

which leaves more room.

This leads to:

```
Sort by end time
```

rather than start time.

This distinction is extremely important.

23. Interval Sorting Rule

A useful interview heuristic:

Merging intervals

Usually:

```
sort by start
```

because you want to process intervals from left to right.

Scheduling / removing overlapping intervals

Often:

```
sort by end
```

because you want to maximize the remaining available space.

Don't memorize this blindly.

Ask:

What ordering makes the greedy decision locally optimal?

PART III — HASHING + INTERVALS

24. Why Combine Hashing and Intervals?

Some problems require both:

```
Range reasoning  
+  
Fast lookup
```

Examples include:

- calendar scheduling
- interval frequency
- event tracking
- coordinate compression
- counting active ranges
- identifying duplicate ranges
- mapping interval endpoints
- sweep-line problems

The hash table stores state while interval logic determines how that state changes.

25. Example — Meeting Rooms

Suppose meetings are:

```
[0, 30]
[5, 10]
[15, 20]
```

Can one person attend all meetings?

Sort by start:

```
[0, 30]
[5, 10]
[15, 20]
```

Check:

```
previous_end = 30
current_start = 5
```

Since:

```
5 < 30
```

there is an overlap.

Therefore, one room/person cannot handle all meetings.

26. Meeting Rooms II

Now ask:

What is the minimum number of rooms required?

This introduces active intervals.

Example:

```
[0, 30]
[5, 10]
[15, 20]
```

At time:

```
0 → 1 active
5 → 2 active
10 → 1 active
15 → 2 active
20 → 1 active
30 → 0 active
```

Maximum active intervals:

```
2
```

Therefore:

```
2 rooms
```

This is a classic sweep-line / heap pattern.

27. Sweep Line Concept

Imagine moving a vertical line across the timeline:

```
0-----5-----10-----15-----20-----30
      ↑
    events
```

At each endpoint:

```
start → +1
end   → -1
```

Maintain:

```
active += event
```

The maximum value of active gives the maximum number of simultaneous intervals.

This generalizes far beyond meetings.

28. Endpoint Hashing

One way to reason about interval events is:

```
time → change in active intervals
```

For example:

```
events = {
  0: +1,
  30: -1,
  5: +1,
  10: -1,
  15: +1,
  20: -1
}
```

Then process endpoints in sorted order.

This gives the mental pattern:

```
Interval
  ↓
Convert to events
  ↓
Hash/map event locations
  ↓
Sort event points
  ↓
Sweep
```

This is often called a difference map / sweep-line technique.

29. Difference Array Connection

Suppose you have:

```
[2,5]
```

Represent its effect as:

```
+1 at 2  
-1 after 5
```

Conceptually:

```
difference[2] += 1  
difference[6] -= 1
```

Then cumulative sum reconstructs the number of active intervals.

This connects:

```
Intervals  
+  
Hash map  
+  
Prefix sum
```

A very powerful combination.

30. Hash Map as a Sparse Difference Array

Suppose the coordinate range is enormous:

```
0 ... 10^9
```

You cannot allocate:

```
difference = [0] * 1_000_000_001
```

Instead store only changed coordinates:

```
difference = {}  
  
difference[start] = difference.get(start, 0) + 1  
difference[end + 1] = difference.get(end + 1, 0) - 1
```

Now:

```
space = O(number of unique endpoints)
```

rather than:

```
O(max_coordinate)
```

This is an important engineering insight.

PART IV — PATTERN RECOGNITION

31. How to Recognize Hashing Problems

Look for phrases such as:

- "find duplicates"
- "have we seen this before?"
- "frequency"
- "count occurrences"
- "two values add to..."
- "first occurrence"
- "last occurrence"
- "group by..."
- "number of times"
- "distinct"
- "subarray sum"
- "previous value"
- "complement"

Immediately consider:

set / dict

32. How to Recognize Interval Problems

Look for:

- meetings
- schedules
- time ranges
- start/end
- events
- overlapping
- merging
- conflicts
- minimum rooms
- maximum simultaneous events
- non-overlapping
- ranges
- bookings
- reservations

Immediately ask:

Should I sort the intervals?

33. Hashing Decision Tree

When you see an array:

Question 1

Do I need to know whether something appeared before?

YES → set

Question 2

Do I need information associated with the value?

YES → dict

Question 3

Do I need counts?

YES → frequency map

Question 4

Do I need previous prefix information?

YES → prefix sum + dict

Question 5

Do I need fast complement lookup?

YES → dict

34. Interval Decision Tree

When you see intervals:

Step 1

Are they sorted?

NO → consider sorting

Step 2

Do I need to combine overlapping intervals?

YES → sort by start + merge

Step 3

Do I need to choose maximum compatible intervals?

YES → consider greedy + sort by end

Step 4

Do I need maximum simultaneous intervals?

YES → sweep line / min heap

Step 5

Do I need interval insertion?

YES → before / overlap / after

35. Common Hashing Mistakes

Mistake 1 — Using a list for membership

Bad for repeated lookups:

```
if x in values:
```

when values is a list.

That lookup is:

$O(n)$

Repeated many times can become:

$O(n^2)$

A set generally provides average:

$O(1)$

membership.

Mistake 2 — Updating before checking

In problems such as Two Sum, order matters.

You often need:

```
check complement  
↓  
then store current value
```

rather than blindly storing first.

Otherwise you can accidentally use the same element twice.

Mistake 3 — Forgetting duplicate frequencies

For counting problems:

`freq[x] = 1`

is insufficient if duplicates matter.

Use:

```
freq[x] = freq.get(x, 0) + 1
```

36. Common Interval Mistakes

Mistake 1 — Forgetting to sort

Trying to merge unsorted intervals directly usually creates unnecessary complexity.

Mistake 2 — Wrong overlap condition

For sorted intervals:

```
current_start <= previous_end
```

usually means overlap under the common closed-interval convention.

Mistake 3 — Using current end instead of maximum

Incorrect:

```
merged[-1][1] = end
```

Correct:

```
merged[-1][1] = max(merged[-1][1], end)
```

because intervals can be nested.

Example:

```
[1,10]  
[2,5]
```

must remain:

```
[1,10]
```

Mistake 4 — Ignoring boundary semantics

Always determine whether:

```
[1,5]  
[5,10]
```

overlap.

The answer depends on the problem.

PART V — HASHING + INTERVALS COMPLEXITY

37. Complexity Cheat Sheet

Pattern • Typical Time • Typical Space
Hash set lookup • $O(1)$ avg • $O(n)$
Frequency map • $O(n)$ • $O(n)$
Two Sum • $O(n)$ avg • $O(n)$
Prefix sum + hash map • $O(n)$ avg • $O(n)$
Sort intervals • $O(n \log n)$ • Depends
Merge intervals • $O(n \log n)$ • $O(n)$ output
Insert interval • $O(n)$ • $O(n)$ output
Interval scheduling • $O(n \log n)$ • $O(1)$ - $O(n)$
Sweep line • $O(n \log n)$ • $O(n)$
Heap-based scheduling • $O(n \log n)$ • $O(n)$

PART VI — GOOGLE INTERVIEW MENTAL MODEL

38. The Most Important Question

Don't ask:

"Which algorithm do I remember?"

Ask:

"What information do I need to remember, and what ordering would make the problem easy?"

This gives two powerful tools:

Remember information
↓
Hashing

and:

Create useful ordering
↓
Sorting

For intervals:

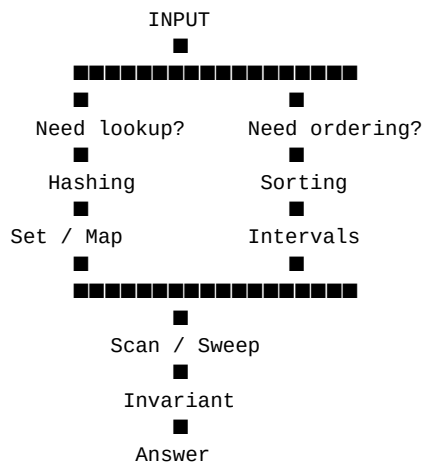
Intervals
↓
Sort
↓
Scan
↓
Maintain invariant

For hashing:

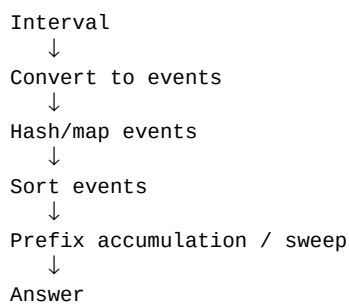
Input
↓
What must I remember?
↓
Set / Map
↓
Constant-average lookup

39. Hashing + Intervals Unified Pattern

A very useful abstraction is:



In more advanced problems:



This is a pattern worth internalizing.

40. Problems You Should Associate With These Patterns

Hashing

Easy → Medium

1. Contains Duplicate
2. Two Sum
3. Valid Anagram
4. Group Anagrams
5. First Unique Character
6. Top K Frequent Elements
7. Longest Consecutive Sequence
8. Subarray Sum Equals K

Core lesson

Value → information

41. Intervals

Core problems to know:

1. Merge Intervals
2. Insert Interval
3. Non-overlapping Intervals
4. Meeting Rooms
5. Meeting Rooms II
6. Minimum Number of Arrows to Burst Balloons
7. Employee Free Time

Core lesson

```
Sort
↓
Scan
↓
Maintain interval invariant
```

42. Hashing + Intervals / Sweep-Line

Progress toward:

1. Meeting Rooms II
2. Car Pooling
3. My Calendar
4. Range Addition
5. Maximum population / active interval problems
6. Difference-array interval problems

These are where you begin combining:

```
Hash Map
+
Sorting
+
Prefix Sum
+
Intervals
```

43. Interview Follow-Ups You Should Practice

For a hashing solution:

Why is lookup $O(1)$?

What is the worst-case complexity?

What is the space complexity?

What happens if memory is constrained?

Can you solve it without extra space?

What happens with duplicate values?

For interval problems:

Why did you sort by start?

Could you sort by end instead?

What is the invariant?

What happens when intervals touch?

What happens when one interval completely contains another?

Can the input be processed online?

How would the solution change for millions of intervals?

How would you handle intervals arriving dynamically?

These follow-ups are often more valuable than merely memorizing the basic implementation.

44. Final Pattern Recognition Cheat Sheet

HASHING

When you hear:

seen
duplicate
frequency
count
lookup
complement
previous
distinct
group
occurrence

Think:

SET / HASH MAP

INTERVALS

When you hear:

- range
- start/end
- meeting
- schedule
- booking
- overlap
- merge
- conflict
- simultaneous

Think:

`SORT + SCAN`

HASHING + PREFIX SUM

When you hear:

- subarray
- sum = k
- number of subarrays
- range sum
- previous cumulative value

Think:

`PREFIX SUM + HASH MAP`

INTERVALS + SWEEP LINE

When you hear:

- maximum simultaneous
- active intervals
- minimum rooms
- events over time
- start/end events

Think:

`EVENTS`
`+`
`SORT`
`+`
`SWEEP`

45. The Four Core Templates to Memorize

Template 1 — Membership

```
seen = set()

for x in nums:
    if x in seen:
        ...
```

```
seen.add(x)
```

Template 2 — Frequency

```
freq = {}

for x in nums:
    freq[x] = freq.get(x, 0) + 1
```

Template 3 — Merge Intervals

```
intervals.sort()

merged = []

for start, end in intervals:
    if not merged or start > merged[-1][1]:
        merged.append([start, end])
    else:
        merged[-1][1] = max(merged[-1][1], end)
```

Template 4 — Prefix Sum + Hash Map

```
prefix = 0
count = 0
seen = {0: 1}

for x in nums:
    prefix += x

    if prefix - k in seen:
        count += seen[prefix - k]

    seen[prefix] = seen.get(prefix, 0) + 1
```

The fourth template is particularly important because it combines two major patterns you've been studying:

```
PREFIX SUM
+
HASHING
↓
SUBARRAY PROBLEMS
```

For your Google preparation, the goal is not to memorize these four code snippets. The goal is to reach the point where, after reading a problem, you can explain:

What information must be remembered? What ordering makes the structure simple? What invariant do I maintain while scanning?

That is the pattern-recognition skill these problems are meant to develop.